Labsheet 8 - Computer vision

Name – Vishavjeet Baraily

3rd year section B

roll no. 68

```
import numpy as np
cap = cv2.VideoCapture("video.mp4")
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
# Shi-Tomasi corner detection
p0 = cv2.goodFeaturesToTrack(
   old_gray,
   maxCorners=100,
   qualityLevel=0.3,
   minDistance=7
)
mask = np.zeros_like(old_frame)
while True:
   ret, frame = cap.read()
   if not ret:
      break
   frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
   # Lucas-Kanade Optical Flow
   p1, status, error = cv2.calcOpticalFlowPyrLK(
      old_gray,
      frame_gray,
      p0,   if p1 is not None:
      good_new = p1[status == 1]
      good_old = p0[status == 1]
      for new, old in zip(good_new, good_old):
         x1, y1 = new.ravel()
         x2, y2 = old.ravel()
         # Draw trajectory
         mask = cv2.line(
            mask,
```

```
            (int(x1), int(y1)),
            (int(x2), int(y2)),
            (0, 255, 0),
            2
        )
        # Draw tracked point
        frame = cv2.circle(
            frame,
            (int(x1), int(y1)),
            4,
            (0, 0, 255),
            -1
        )
    output = cv2.add(frame, mask)
    cv2.imshow("Object Tracking", output)
    old_gray = frame_gray.copy()
    p0 = good_new.reshape(-1, 1, 2)

  if cv2.waitKey(30) & 0xFF == 27:
    break
cap.release()
cv2.destroyAllWindows()
```

Note: Replace "video.mp4" with the name/path of your video. For a webcam, use:

```
cap = cv2.VideoCapture(0)
```

Farneback Dense Optical Flow

```
import cv2
import numpy as np
cap = cv2.VideoCapture("video.mp4")
ret, old_frame = cap.read()
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
while True:
  ret, frame = cap.read()
  if not ret:
    break
```

```
    new_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        new_gray,
        None,
        0
    )
    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )
    hsv = np.zeros_like(frame)
    hsv[..., 0] = angle * 180 / np.pi / 2
    hsv[..., 1] = 255
    hsv[..., 2] = cv2.normalize(
        magnitude, None, 0, 255, cv2.NORM_MINMAX
    )
    output = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
    cv2.imshow("Dense Optical Flow", output)
    old_gray = new_gray.copy()

    if cv2.waitKey(30) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Observations

Condition	Observation
Normal movement	Feature points follow the object's movement.
Fast movement	Tracking accuracy may decrease.
Lighting change	Feature tracking can become unstable.
Occlusion	Some feature points may be lost.
Camera movement	Motion may appear across the entire frame.

Condition	Observation
Lucas-Kanade	Tracks selected feature points efficiently.
Farneback	Provides dense motion information for the complete image.

Result

The optical-flow-based real-time object tracking system was successfully implemented using Shi-Tomasi Corner Detection, Lucas-Kanade Optical Flow, and Farneback Dense Optical Flow. Object trajectories and motion patterns were successfully visualized.

Questions and Answers

1. How does object tracking differ from object detection?

Object Detection identifies the location of an object in each frame.

Object Tracking follows the same object across multiple consecutive frames.

For example, detection finds a car, while tracking continuously follows that car as it moves.

---

2. Explain how Optical Flow can be used for real-time object tracking.

Optical Flow calculates the movement of pixels between consecutive frames. Feature points are selected from an object and their new positions are calculated in the next frame.

By continuously updating these positions, the trajectory and movement of the object can be tracked in real time.

---

3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Shi-Tomasi Corner Detection identifies strong and easily trackable feature points, such as corners, in the image.

Lucas-Kanade then tracks these points between consecutive frames to estimate their movement.

Thus:

Shi-Tomasi → selects feature points
Lucas-Kanade → tracks feature points

---

4. Why is Optical Flow suitable for motion analysis in videos?

Optical Flow directly estimates the movement of pixels between consecutive frames. Therefore, it provides information about:

- **Direction of movement**
- **Speed/magnitude of movement**
- **Object trajectory**
- **Overall scene motion**

This makes it useful for analyzing dynamic video scenes.

---

5. What challenges arise while tracking fast-moving or partially occluded objects?

Fast-moving objects can move significantly between two frames, making feature matching difficult. Partial occlusion can hide important feature points.

Other problems include:

- **Motion blur**
- **Lighting changes**
- **Camera movement**
- **Loss of feature points**

---

6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Optical Flow	Deep Learning Tracking
Uses pixel motion between frames.	Uses trained neural networks.
Generally requires less computational power.	Usually requires more computational resources.
Does not require large training datasets.	Requires training data/models.
Useful for motion-based tracking.	Can identify and track specific object classes.
Can be affected by lighting and occlusion.	Often more robust to complex scenes.

---

7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

In traffic monitoring, Optical Flow can estimate vehicle movement, direction, and speed.

In autonomous driving, it can help estimate:

- **Vehicle and pedestrian movement**
- **Camera motion**
- **Relative motion of objects**
- **Possible obstacles**

---

8. Explain the effect of camera motion on Optical Flow estimation.

When the camera moves, many or all pixels in the scene appear to move. Optical Flow may therefore interpret camera movement as object movement.

Camera motion can be reduced or compensated using techniques such as camera-motion estimation and image stabilization.

---

9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

1. **Traffic monitoring**
2. **Video surveillance**
3. **Autonomous vehicles**
4. **Robotics**